

Uso de la Biblioteca Pomegranate en Probabilidad y Ciencia de Datos

Brayan David Santos Alvarez Código: 20251020157 Materia: Probabilidad y Estadística

25 de febrero de 2026

1. Introducción

La biblioteca **pomegranate** es una herramienta de Python orientada al modelado probabilístico y aprendizaje automático basado en modelos estadísticos. Permite trabajar con distribuciones de probabilidad, modelos ocultos de Markov, mezclas gaussianas y redes bayesianas.

Este documento describe las principales propiedades y funciones de sus componentes más importantes y explica un ejemplo práctico relacionado con la probabilidad de aprobación de un estudiante.

2. Componentes Principales de la Biblioteca

2.1. Distribuciones Discretas (DiscreteDistribution)

Permiten modelar variables aleatorias discretas.

Propiedades principales:

- Diccionario de probabilidades asociadas a cada valor.
- Debe cumplir que la suma de probabilidades sea igual a 1.

Funciones principales:

- `sample()`: genera muestras aleatorias.
- `probability()`: calcula la probabilidad de un valor.

2.2. Tabla de Probabilidad Condicional (ConditionalProbabilityTable)

Modela probabilidades condicionadas a otras variables.

Propiedades:

- Define combinaciones padre-hijo.
- Cada combinación debe sumar 1 respecto a la variable dependiente.

Uso principal: Permite aplicar probabilidad condicional y modelar dependencias entre variables.

2.3. Estados (State)

Representan nodos dentro de una red bayesiana.

Funciones:

- Asociar una distribución a un nodo.
- Permitir conexiones mediante aristas.

2.4. Red Bayesiana (BayesianNetwork)

Modelo gráfico probabilístico que representa dependencias entre variables.

Funciones principales:

- `addstates()` : *agreganodos*.
- `addedge()` : *crearelacionesdedependencia*.
- `bake()`: compila el modelo.
- `predictproba()` : *realizainferenciaprobabilística*.
- `sample()`: genera simulaciones.

3. Ejemplo Práctico: Aprobación de un Estudiante

3.1. Código Completo del Programa

A continuación se presenta el código original desarrollado en Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pomegranate import *

# Distribuciones iniciales

estudio = DiscreteDistribution({"Bajo": 0.4, "Alto": 0.6})
asistencia = DiscreteDistribution({"Baja": 0.3, "Alta": 0.7})
dificultad = DiscreteDistribution({"Fácil": 0.5, "Difícil": 0.5})

# Tabla de probabilidad condicional

aprueba = ConditionalProbabilityTable([
["Bajo","Baja","Fácil","Sí",0.4],
["Bajo","Baja","Fácil","No",0.6],
["Bajo","Baja","Difícil","Sí",0.1],
["Bajo","Baja","Difícil","No",0.9],
["Bajo","Alta","Fácil","Sí",0.6],
["Bajo","Alta","Fácil","No",0.4],
```

```

["Bajo", "Alta", "Difícil", "Sí", 0.2],
["Bajo", "Alta", "Difícil", "No", 0.8],
["Alto", "Baja", "Fácil", "Sí", 0.7],
["Alto", "Baja", "Fácil", "No", 0.3],
["Alto", "Baja", "Difícil", "Sí", 0.4],
["Alto", "Baja", "Difícil", "No", 0.6],
["Alto", "Alta", "Fácil", "Sí", 0.95],
["Alto", "Alta", "Fácil", "No", 0.05],
["Alto", "Alta", "Difícil", "Sí", 0.7],
["Alto", "Alta", "Difícil", "No", 0.3]
], [estudio, asistencia, dificultad])

# Creación de estados

s1 = State(estudio, name="Estudio")
s2 = State(asistencia, name="Asistencia")
s3 = State(dificultad, name="Dificultad")
s4 = State(aprueba, name="Aprueba")

# Red bayesiana

modelo = BayesianNetwork("Modelo Aprobación")
modelo.add_states(s1, s2, s3, s4)
modelo.add_edge(s1, s4)
modelo.add_edge(s2, s4)
modelo.add_edge(s3, s4)
modelo.bake()

# Inferencia

resultado = modelo.predict_proba({
    "Estudio": "Alto",
    "Asistencia": "Alta",
    "Dificultad": "Difícil"
})
print(resultado[3].parameters)

# Simulación

datos = modelo.sample(1000)
df = pd.DataFrame(datos, columns=["Estudio", "Asistencia", "Dificultad", "Aprueba"])

prob_aprueba = df["Aprueba"].value_counts(normalize=True)
print(prob_aprueba)

prob_aprueba.plot(kind="bar")
plt.title("Probabilidad de Aprobar vs No Aprobar")
plt.show()

```

3.2. Explicación Paso a Paso

1. Importación de librerías Se importan bibliotecas para cálculo numérico (NumPy), manejo de datos (Pandas), visualización (Matplotlib) y modelado probabilístico (pomegranate).

2. Definición de distribuciones iniciales Se modelan las probabilidades a priori de Estudio, Asistencia y Dificultad. Por ejemplo:

$$P(\text{Estudio} = \text{Alto}) = 0,6 \quad (1)$$

Estas representan probabilidades marginales.

3. Tabla de probabilidad condicional Aquí se define:

$$P(\text{Aprueba} \mid \text{Estudio}, \text{Asistencia}, \text{Dificultad}) \quad (2)$$

Cada combinación de variables independientes tiene asociada una probabilidad de aprobar o no aprobar.

4. Construcción de la red Se crean nodos (State) y se conectan mediante aristas dirigidas hacia la variable Aprueba. Luego se ejecuta `bake()` para compilar el modelo.

5. Inferencia Se calcula la probabilidad de aprobar bajo condiciones específicas: Estudio = Alto, Asistencia = Alta, Dificultad = Difícil.

El resultado esperado es aproximadamente:

$$P(\text{Aprueba} = \text{Sí} \mid \text{Alto}, \text{Alta}, \text{Difícil}) = 0,7(3)$$

6. Simulación Monte Carlo Se generan 1000 muestras aleatorias del modelo para estimar probabilidades empíricas usando frecuencias relativas.

3.3. Análisis de Resultados

Del modelo teórico se observa que:

- Cuando el nivel de estudio es alto y la asistencia es alta, la probabilidad de aprobar es muy elevada.
- Cuando el examen es difícil y el estudio es bajo, la probabilidad disminuye considerablemente.
- El estudio tiene mayor impacto que la asistencia cuando el examen es difícil.

En la simulación de 1000 datos, la proporción de aprobados suele estar cercana al valor esperado teórico global, lo cual valida el modelo probabilístico.

La gráfica de barras permite visualizar claramente la diferencia entre aprobar y no aprobar.

4. Conclusiones

El modelo demuestra cómo las redes bayesianas permiten combinar probabilidades marginales y condicionales para analizar fenómenos reales con incertidumbre.

El uso de simulación Monte Carlo confirma los resultados teóricos mediante datos generados artificialmente, integrando probabilidad y ciencia de datos en un mismo análisis.

La biblioteca pomegranate es una herramienta poderosa para modelar sistemas probabilísticos. Permite aplicar conceptos de probabilidad condicional, inferencia bayesiana y simulación Monte Carlo dentro de un entorno de ciencia de datos.

El ejemplo desarrollado evidencia la utilidad de las redes bayesianas para analizar factores que influyen en eventos inciertos como la aprobación académica.